

AfriAgriFed

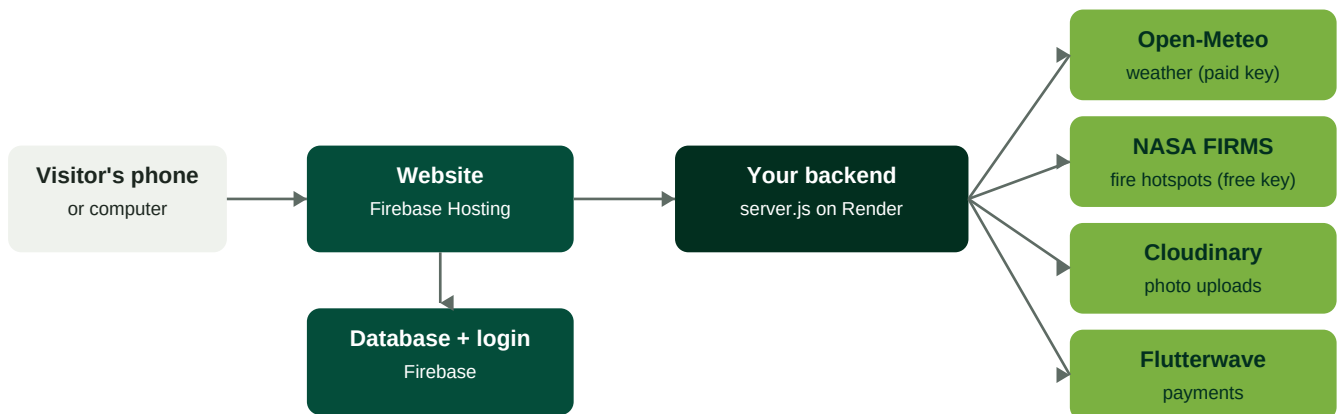
Setup guide: API keys, accounts
and servers, step by step

Written for the person doing this for the first time. No programming needed. Every step says what to click, what to type, and how to check that it worked.

Read this first

This guide takes AfriAgriFed from "works on my computer" to "works for everyone". There are three jobs:

- **Get the keys.** A key is a long secret password that lets our server use another company's service (weather, fire data, payments, photos).
- **Put the backend online.** The backend is the small program in `server.js`. It holds the keys, keeps one shared copy of the weather and fire data so every visitor loads instantly, and processes payments and uploads. Today it only runs on your computer, so nobody else can use it.
- **Connect the website to it,** then deploy.



Visitors only ever talk to the website and your backend. Your backend talks to the other services. The keys never reach a visitor's browser.

The three rules for keys

1. A key lives in only two places: the file `.env` on your computer, and the "Environment" page of your hosting dashboard.
2. Never paste a key into a chat, an email, a screenshot, a document, or any file that goes to GitHub. `.env` is already excluded from GitHub; keep it that way.
3. If a key is ever exposed, treat it as stolen: create a new one in that service's dashboard, update both places, and delete the old one.

How long it takes

Job	Time	Waiting on someone else?
Open-Meteo commercial plan	15 minutes	Possibly (you may need to email them for a quote)
NASA FIRMS fire key	5 minutes	No: the key arrives by email within minutes
Firebase admin key	10 minutes	No
Hosting the backend (Render)	30 minutes	No
Connect, build, deploy the website	20 minutes	No
EUMETSAT satellite permission	A few days or more	Yes: you are asking their permission

1. What you need: the checklist

Tick these off as you go. "Name in .env" is the exact label the program looks for; spelling and capital letters matter.

#	What	Why	Needed to launch?	Name in .env
1	Open-Meteo commercial plan and key	All weather: wind, rain, temperature, forecasts, flood outlook	Yes. The free tier forbids commercial use	OPEN_METEO_API_KEY
2	NASA FIRMS map key	Fire hotspot layer and farm fire alerts	No: the fire layer simply stays hidden without it	FIRMS_MAP_KEY
3	Cloudinary keys	Photo and PDF uploads	Yes (you may already have these)	CLOUDINARY_CLOUD_NAME CLOUDINARY_API_KEY CLOUDINARY_API_SECRET
4	Flutterwave keys	Payments and seller payouts	Yes	FLW_SECRET_KEY FLW_PUBLIC_KEY FLW_ENCRYPTION_KEY FLW_SECRET_HASH
4 b	Firebase admin key (service account)	Lets the backend price orders, check who is buying and record paid orders safely	Yes: payments stay OFF without it	FIREBASE_SERVICE_ACCOUNT_JSON
5	Render account	Puts the backend online 24/7	Yes (or any similar host)	(no key)
6	Firebase project	Website hosting, database, login (already set up)	Already done	(project afriagrifed-ebc30)
7	EUMETSAT permission	Live satellite clouds and lightning on the map	Yes for commercial use, or hide those layers	(a letter, not a key)
8	Map tile provider (later)	Production-grade background map	Recommended. Not wired in yet	REACT_APP_MAPTILER_KEY
9	Email provider (later)	Welcome and notification emails	Not built yet	EMAIL_API_KEY

Why Open-Meteo costs money

AfriAgriFed takes a commission on sales, so it counts as a commercial service. Open-Meteo's free plan is for non-commercial use only. Their commercial plans use a separate address and include a commercial licence. The backend switches to that address automatically as soon as it finds your key.

2. Getting each key

2.1 Open-Meteo (weather)

1. Open open-meteo.com/en/pricing in your browser.
2. Choose the **API Standard** plan (1 million calls a month). Their page lists API Standard, API Professional and API Enterprise, but does not publish the price: use the contact option and ask for a quote. You will get an email once the plan is active.

3. When the plan is active you will be given an **API key**: a long string of letters and numbers. Copy it somewhere safe for the next 10 minutes (you will paste it into `.env` and into Render, then delete the copy).
4. Tell them (or check their terms) that you are a commercial agri-marketplace in Namibia, so there is no doubt about your licence.

How many calls will we use?

Because the backend keeps one shared copy, the national forecast grid is fetched once an hour for everybody. Open-Meteo counts each location in a request as a call (confirm this on their terms page); 195 locations x 24 hours x 30 days is about 140,000 a month, well inside 1 million. Taps on the map are cached for 30 minutes per 10 km square, so traffic growth barely changes the bill.

One thing to check after you get the key

Their page says the paid plan serves every service, including the river flood outlook, from `customer-api.open-meteo.com`. If a flood outlook ever stops appearing on the map after you switch to the paid key, send Open-Meteo support the address of the flood endpoint your plan should use; the map still works without it.

2.2 NASA FIRMS (fire hotspots), free

1. Go to `firms.modaps.eosdis.nasa.gov/api/map_key`.
2. Enter your email address and submit. NASA emails you a **MAP_KEY** (about 32 characters).
3. That is the whole key. It allows 5,000 requests every 10 minutes; our server makes about 8 an hour.

What the fire layer does and does not mean

A hotspot is a satellite seeing heat. It is not a confirmed fire, and a satellite passes only a few times a day, so an empty map is not a guarantee of safety. The app says this on screen every time. Please keep that wording if you change the page.

2.3 Cloudinary (photo uploads)

1. Sign in at `cloudinary.com`.
2. Click the **gear icon** (Settings), then **API Keys**.
3. Copy the **Cloud name**, **API Key** and **API Secret**. These are the three Cloudinary lines in `.env`.

2.4 Flutterwave (payments)

1. Sign in at `dashboard.flutterwave.com`.
2. **Start in Test mode** (the switch at the top of the dashboard). Go to **Settings, then API Keys**. Copy the test **Secret key**, **Public key** and **Encryption key**.
3. Go to **Settings, then Webhooks**. Make up a long random phrase (for example 30 random letters and numbers) and enter it as the **Secret hash**. Use the *same* phrase as `FLW_SECRET_HASH` in the backend. The webhook address comes in section 5.
4. Make several test purchases (Flutterwave publishes test card numbers in its documentation). Only when everything works, switch the dashboard to **Live**, copy the live keys, and replace the test keys.

How orders are recorded now (good news)

The backend, not the buyer's browser, decides the price, pays the seller's own account, and records the order once Flutterwave confirms the charge. A buyer can no longer change a price or invent an order. For this the backend needs the Firebase admin key in section 2.5, and until it has it, **it refuses to start any payment** (the buyer sees "Payments are being set up") rather than take money it cannot record.

2.5 Firebase admin key (so the backend can record orders)

1. Open `console.firebase.google.com` and choose your project (`afriagrifed-ebc30`).
2. Click the **gear icon** next to Project Overview, then **Project settings**, then the **Service accounts** tab.
3. Click **Generate new private key**, then **Generate key**. A small file ending in `.json` downloads. **This file is a master key to your database.**
4. Turn it into one line of text that is easy to paste. Open **PowerShell** and type the following, changing the path to where the file was saved. It copies the result to your clipboard; nothing is printed:

```
$file = "$env:USERPROFILE\Downloads\your-key-file.json"
[Convert]::ToBase64String([IO.File]::ReadAllBytes($file)) | Set-Clipboard
```

5. Paste that text as the value of `FIREBASE_SERVICE_ACCOUNT_JSON` on Render (section 4.2), and in your local `.env` if you run the backend on your computer.
6. Then **delete the downloaded .json file** (or move it to a locked folder, never a shared one). Never email it, upload it, or put it in the project folder's Git history.

If the key ever leaks

In the same Service accounts tab, open the key's menu in Google Cloud and delete it, generate a new one, and update Render. The old key stops working immediately.

2.6 EUMETSAT (satellite clouds and lightning): permission, not a key

1. The map's cloud, storm and lightning layers use free satellite imagery from EUMETSAT. Their data is free for personal or non-commercial use; other use needs their authorisation.
2. Find the user helpdesk contact on `eumetsat.int`. Write a short email: who you are, what AfriAgriFed does, that you display Meteosat-11 and MTG imagery (credited as EUMETSAT) on a map for Namibian farmers, and ask for written permission for commercial display.
3. Keep their reply. Until you have it, you can hide those layers: in `src/farmview/config.js` remove the entries with `kind: "sat"`.

3. Putting the keys in your .env file (on your computer)

1. Open the folder `D:\Desktop\afriagrifed-main`.
2. Find the file called `.env`. (If you cannot see it: in File Explorer choose View, then Show, then Hidden items. If it does not exist, copy `.env.example` and rename the copy to `.env`.)
3. Right-click it, choose **Open with**, then **Notepad**.
4. After each = paste the key with **no spaces and no quotes**. Save.
5. If the program is running, stop it (**Ctrl+C** in its window) and start it again. It only reads the file when it starts.

```
# What the new lines look like (these are made-up examples, not real keys)
OPEN_METEO_API_KEY=abcd1234efgh5678
FIRMS_MAP_KEY=0123456789abcdef0123456789abcdef
FIREBASE_SERVICE_ACCOUNT_JSON=eyJ0eXBlijoic2VydmVjZV9hY2NvdW50Ii4uLg==
CORS_ORIGINS=https://afriagrifed-ebc30.web.app,https://afriagrifed-ebc30.firebaseio.com
TRUST_PROXY=1
RATE_LIMIT_PER_MINUTE=120
```

To run the backend on your own computer (for testing), open a command window in the project folder and type:

```
node server.js
```

It should print *Server running on port 5000*. If you have no Open-Meteo key it will say it is using the free, non-commercial tier, and if you have no FIRMS key it will say the fire layer is switched off. Both messages are

normal in testing.

Only names starting with `REACT_APP_` are public

Anything in `.env` that starts with `REACT_APP_` is baked into the public website. Never put a secret on such a line. The weather, fire, Cloudinary and Flutterwave keys have no such prefix, so they stay on the server.

4. Putting the backend online (Render)

Render is a hosting company that runs programs like `server.js` for you, with a free web address. Any similar host (Railway, Fly.io) works the same way with the same commands. Render is described here because its screens are the simplest.

4.1 Create the service

1. Go to render.com and sign up with your **GitHub** account (this lets Render see your code).
2. Click **New +**, then **Web Service**. Pick the repository `D0MINVS98/afriagrfd`.
3. **Branch**: use `main` once you have merged your work; until then choose `ui-overhaul-and-commerce`.

Setting	Type exactly
Name	afriagrfd-api (this becomes part of your web address)
Region	Frankfurt (the closest Render region to Namibia)
Runtime / Language	Node
Build Command	npm install
Start Command	node server.js
Instance type	Free to try (it falls asleep after a quiet spell and takes about a minute to wake up). For real customers choose a paid instance so it is always awake; see Render's pricing page for the current price.
Health Check Path	/api/health

4.2 Add the environment variables

On the same page find **Environment Variables** (or open the service, then **Environment**). Click **Add Environment Variable** for each row. The values are the ones from sections 2 and 3. Copy them from your `.env`; do not type them into any chat.

Name	Value	Required?
NODE_VERSION	20	Recommended
OPEN_METEO_API_KEY	your Open-Meteo key	Yes for launch
FIRMS_MAP_KEY	your NASA FIRMS key	For the fire layer
CLOUDINARY_CLOUD_NAME CLOUDINARY_API_KEY CLOUDINARY_API_SECRET	from Cloudinary	Yes (the server will not start without them)
FLW_SECRET_KEY FLW_PUBLIC_KEY FLW_ENCRYPTION_KEY FLW_SECRET_HASH	from Flutterwave (test keys first)	Yes for payments

Name	Value	Required?
FIREBASE_SERVICE_ACCOUNT_JSON	the one line of text from section 2.5	Yes for payments: without it the backend refuses to take any payment
PLATFORM_COMMISSION_RATE	0.05	Optional (5% default)
APP_BASE_URL	https://afriagrifed-ebc30.web.app (your default Firebase address: confirm it in the Firebase console under Hosting; or use your own domain)	Yes: payment redirects use it
CORS_ORIGINS	https://afriagrifed-ebc30.web.app,https://afriagrifed-ebc30.firebaseio.com (add your own domain too, separated by commas)	Yes: locks the backend to your website
TRUST_PROXY	1	Yes on Render: lets rate limiting see each visitor
RATE_LIMIT_PER_MINUTE	120	Optional

4.3 Start it and check it

1. Click **Create Web Service**. Render builds it; this takes a few minutes. Wait for the word **Live**.
2. Copy your web address from the top of the page. It looks like `https://afriagrifed-api.onrender.com`.
3. Open `https://afriagrifed-api.onrender.com/api/health` in your browser. You should see `{"ok":true,...}`.
4. Open `.../api/features`. You should see `"commercialweather":true` and `"fires":true` if the keys are set. If either says false, that key is missing or has a typo.
5. Open `.../api/weather/grid`. After a few seconds you should see a large block of numbers (the national forecast). Reload it: it should now appear instantly. That is the shared cache working.

Free plan note

Render's free instances forget their temporary files when they sleep. That is harmless here (the weather is simply fetched again), but it is another reason to use a paid instance for real customers. Whichever you use, the backend refreshes the weather in the background every 50 minutes so visitors never wait for it.

5. Connecting the website to the backend

Do these in this order, or purchases will stop working

The new database rules stop browsers from creating orders themselves, because the backend does it now. So: **(1)** get the backend online with ALL its keys including the Firebase admin key (sections 2 to 4) and check `/api/features` says `"payments":true`; **(2)** only then build and deploy the website and rules below. If you deploy the rules first, buyers will pay nothing and see an error, which is safe but looks broken.

1. Open `.env` (see section 3) and set the address of your backend. It must start with `https://` and have no slash at the end:

```
REACT_APP_API_URL=https://afriagrifed-api.onrender.com
```

2. *Optional, once you have the paid Open-Meteo key:* add the line below. It forbids the website from ever falling back to Open-Meteo's free service if your backend is down. Leave it out while testing.

```
REACT_APP_WEATHER_DIRECT_FALLBACK=false
```

3. Open a command window in the project folder and build the website:

```
npm run build
```

4. Deploy the website and the database rules. The first time only, sign in to Firebase (a browser window opens; use the Google account that owns the project):

```
npx firebase-tools login
```

Then deploy (this is one line; note the "i" in afriagrified, which differs from the folder name):

```
npx firebase-tools deploy --only "hosting,firestore:rules" --project afriagrified-ebc30
```

If npx cannot download Firebase's tool on your connection

A copy is already saved on this computer. In a command window, type the following as ONE line (it is long):

```
cd /d D:\Desktop\afriagrified-main && node C:\Users\lamek\AppData\Local\npm-cache\_npx\7750544ccf494d8b\node_modules\firebase-tools\lib\bin\firebase.js deploy --only "hosting,firestore:rules" --project afriagrified-ebc30
```

5. Open your live website, press **Ctrl+Shift+R** to force a fresh copy, and go to **Map**.
6. Check it uses your backend: press **F12**, open the **Network** tab, reload, and look for requests to your Render address (`/api/weather/grid` and `/api/features`). If you see requests to `api.open-meteo.com` instead, the website was built before you set `REACT_APP_API_URL`; rebuild and redeploy.
7. Under **Hazards** in the map's left panel there should now be a **Fire hotspots** button (it only appears when the backend has a FIRMS key).

6. The Flutterwave webhook

1. In the Flutterwave dashboard go to **Settings, then Webhooks**.
2. Set the **URL** to your backend address followed by `/api/payments/webhook`, for example `https://afriagrified-api.onrender.com/api/payments/webhook`.
3. Make sure the **Secret hash** there is exactly the same phrase as `FLW_SECRET_HASH` on Render. If they differ, the backend rejects every notification.
4. Save. When a payment succeeds, Flutterwave tells your backend straight away, and the backend records the order even if the buyer closed their browser before returning to the site. If the buyer does return, the page simply shows it confirmed. Either way each payment becomes exactly one order.

7. Is everything working? A final check

Check	Where to look	You should see
Backend alive	https://YOUR-BACKEND/api/health	<code>{"ok":true,...}</code>
Keys detected	https://YOUR-BACKEND/api/features	"commercialWeather":true, "fires":true, "payments":true (payments is true only when BOTH the Flutterwave key and the Firebase admin key are set)
Weather cached	https://YOUR-BACKEND/api/weather/grid (twice)	Numbers. Second load is instant
Fires	https://YOUR-BACKEND/api/fires	A list of hotspots (it can be empty on a quiet day). If it says "fires_not_configured" the key is missing
Website uses backend	Live site, Map, F12, Network	Requests to your backend address
Weather colours and wind	Map page	Animated wind and the Rain/Temperature buttons work
Fire layer	Map, Hazards, Fire hotspots	Coloured dots; tap near one and read the "fire hotspots within 25 km" note
Farmer fire alerts	Sign in as a farmer with a farm saved	A red or amber note under My farms when a hotspot is within 25 km
Photo upload	Post a photo on the Community Feed	The photo appears
Payment (test mode)	Buy a listing with a test card	Returns to the site, offers "Track your order", and the order appears in the Order Tracker once
Price cannot be changed	Try buying as a second test buyer	Both are charged the listing's price; the seller gets a notification each time
Seller payouts	A seller opens My Listings and saves bank details	"Payouts are set up". Listings can now be bought

8. When something goes wrong

What you see	Likely cause	What to do
Map weather loads slowly the first time, then fast	Free Render plan waking up	Normal on the free plan. Use a paid instance for customers
Browser console says "blocked by CORS policy"	Your website address is not in CORS_ORIGINS	Add the exact address (https, no slash at the end) to CORS_ORIGINS on Render, save, wait for it to redeploy
No Fire hotspots button	No FIRMS_MAP_KEY on the backend, or the website was built without REACT_APP_API_URL	Check /api/features. Set the key and redeploy; or rebuild the website
/api/features says commercialWeather false	OPEN_METEO_API_KEY missing or has a space or quote around it	Re-paste on Render with no spaces; Save; it redeloys
Weather suddenly stops after you add the key	Key not yet active, or wrong	Open Render's Logs. A line starting [weather] shows the provider's status code (401 or 403 means the key is wrong)
Map says "Weather data is temporarily unavailable"	Open-Meteo is down or your plan is exhausted	The backend keeps showing the last good copy for up to 12 hours. Check Open-Meteo's status and your plan
Error 429 "Too many requests"	One visitor made over 120 requests a minute	Normal protection. Raise RATE_LIMIT_PER_MINUTE only if real users are being blocked
Buyers see "Payments are being set up"	The backend has no Firebase admin key, or it could not be read (so it refuses to take money)	Check /api/features shows payments true. In Render's Logs look for "Could not read the Firebase service account" or "payments are switched OFF". Re-paste the key (section 2.5)
A buyer paid but no order appears	The webhook is not reaching the backend	Check the webhook address and Secret hash (section 6). The buyer's return page also creates it; ask them to open the link Flutterwave emailed, or reload /payment-callback
Payments send people to localhost	APP_BASE_URL not set on Render	Set it to your website address
Uploads fail	Cloudinary keys missing on Render, or REACT_APP_API_URL wrong	Check the three Cloudinary values and the backend address
Render deploy fails	Missing Cloudinary variables (the server stops on purpose)	Open Render's Logs; the first red line names what is missing

9. Words used in this guide

Word	Plain meaning
Backend	A small program (server.js) running on a computer that is always on. The website asks it for things; it holds the secrets.
API	The doorway a service offers so programs can ask it for data, for example "give me the forecast for Namibia".
API key / token	A secret password that identifies you to a service and lets them charge or limit you.
Environment variable	A named setting (like OPEN_METEO_API_KEY) given to a program from outside, so secrets never sit in the code.
CORS	A browser safety rule. CORS_ORIGINS lists which websites may use your backend.
Cache	A saved copy reused for a while, so ten thousand visitors cause one request to the weather company, not ten thousand.
Webhook	A message a service sends to your backend when something happens (for example a payment succeeded).
Hotspot	A satellite detection of heat on the ground. A sign of possible fire, not proof.

10. Costs and licences at a glance

Prices change, and several providers do not publish them, so this guide gives none. Check each provider's pricing page on the day you sign up.

Item	Cost type	Licence note
Open-Meteo API Standard	Monthly plan (quote)	Includes commercial use. Credit "Weather data by Open-Meteo.com" (shown on the map)
NASA FIRMS	Free	Free for everyone. Credit NASA FIRMS (shown on the map)
Render	Free to try, paid for always-on	No licence issue
Cloudinary	Free tier, then by usage	Check the free tier's limits against expected photo volume
Flutterwave	A percentage per payment	Seller payouts need each seller's subaccount; AfriAgriFed keeps the commission rate you set
Firebase	Free tier, then by usage	Watch the database read counts as users grow
EUMETSAT imagery	Free	Non-commercial unless authorised: see 2.6
Esri and OpenStreetMap map tiles	Free	Fine for testing. For launch move to a licensed tile provider (such as MapTiler); ask me to wire it in once you have a key

Please also do before launch

Review the crop thresholds with agricultural extension officers, add a privacy policy that mentions location and farm records, test the Firestore rules and the full buy-and-sell flow in a staging project with Flutterwave test keys. The full list is in docs/LAUNCH_CHECKLIST.md.